4CS017

Internet Software Architecture and Database

Portfolio/Report

Name: - **Rohit Sahani**

Student Id: **-NP02CS4A250027**

Prototype-1

Reflective Report on Weather App Architecture

The Weather App is designed using a client-server architecture, where the front-end interacts with the OpenWeatherMap API to fetch and display weather data. The app is simple, user-friendly, and provides real-time weather information. Using async/await ensures that API calls do not block the interface, maintaining smooth user interaction. Separating HTML, CSS, and JavaScript improves code clarity and maintainability. The app also includes a default city (Biratnagar), ensuring that users see immediate information without input.

The Activity Diagram illustrates the workflow: the user inputs a city, clicks the search button, the app fetches data from the API, and displays the results. The Sequence Diagram shows interactions over time: the user triggers a search, the getWeather function fetches data, parses it, and updates the DOM. The Deployment Diagram highlights that the app runs on a web browser and communicates with the remote API server via the internet. These diagrams help visualize the structure, flow, and dependencies of the application.

The app's main strengths are simplicity, modularity, and responsiveness. It is easy to maintain and extend for small features. Error handling for invalid city input provides basic user feedback. The use of asynchronous calls ensures that the app remains responsive during data fetches.

However, there are notable weaknesses. The app relies entirely on the API; network issues or server downtime prevent it from functioning. Error messages are minimal, and the API key is exposed in client-side code, creating potential security risks. The app is also not scalable, lacking features like caching, multi-city comparison, or offline support.

In conclusion, the Weather App has a clear, simple, and effective architecture for lightweight applications. Its design ensures good user experience and maintainability, but improvements in security, error handling, and scalability would enhance its robustness and future readiness.

Prototype-2

Reflective Report: Weather App with Server-Side Caching

This weather app is made using PHP and MySQL to keep track of the weather in cities. It does this so that it does not have to keep asking the OpenWeatherMap API for the information over and over again. When you look for a city, the app checks the database first. If the information in the database is not than 10 minutes old it will show you that instead of getting new information. This weather app works faster because of this. It also helps reduce the work the server has to do. The weather app is really, about getting the city weather data and the city weather data is what it focuses on.

The app works in three parts: the front part users see, the back part that talks to the weather service, and the database that saves data. When the app starts, it creates the database and table if they don't exist, making setup easy. Users see the default city right away and can search for others.

A big plus is speed—returning users see fast results for popular cities. The clean design also makes it easy to add new features later. However, the 10-minute cache may show slightly old weather. Also, the code does not use prepared statements, which could let bad input harm the database if not cleaned well.

The app runs on one server, so it might slow down with many users. It also depends fully on the OpenWeatherMap service—if it fails, the app can't get new data unless old data is cached. Error handling is basic, with no retry plan.

While the app works well for small use, it could be safer and more stable with better security, smarter caching, and backup plans for outages. Using prepared statements would protect the database. A shorter or adjustable cache time could improve accuracy. Adding more servers or using cloud tools could help it grow. Still, for now, it gives users quick weather info with less strain on the system.

Prototype-3

Reflective Report: Weather Application Prototype 3

This report looks at the strengths and weaknesses of the latest weather application prototype. The app is built using HTML, CSS, and JavaScript for the visual part, and PHP with a MySQL database for handling data. The goal of this version was to create a fast, professional tool that provides accurate weather information.

One of the biggest strengths of this application is **speed and efficiency**. By using "caching," the app saves weather data in two places: the user's web browser and the server's database. This means if a user searches for the same city twice, the app doesn't have to ask the weather provider for the data again. This makes the app load much faster and saves API costs.

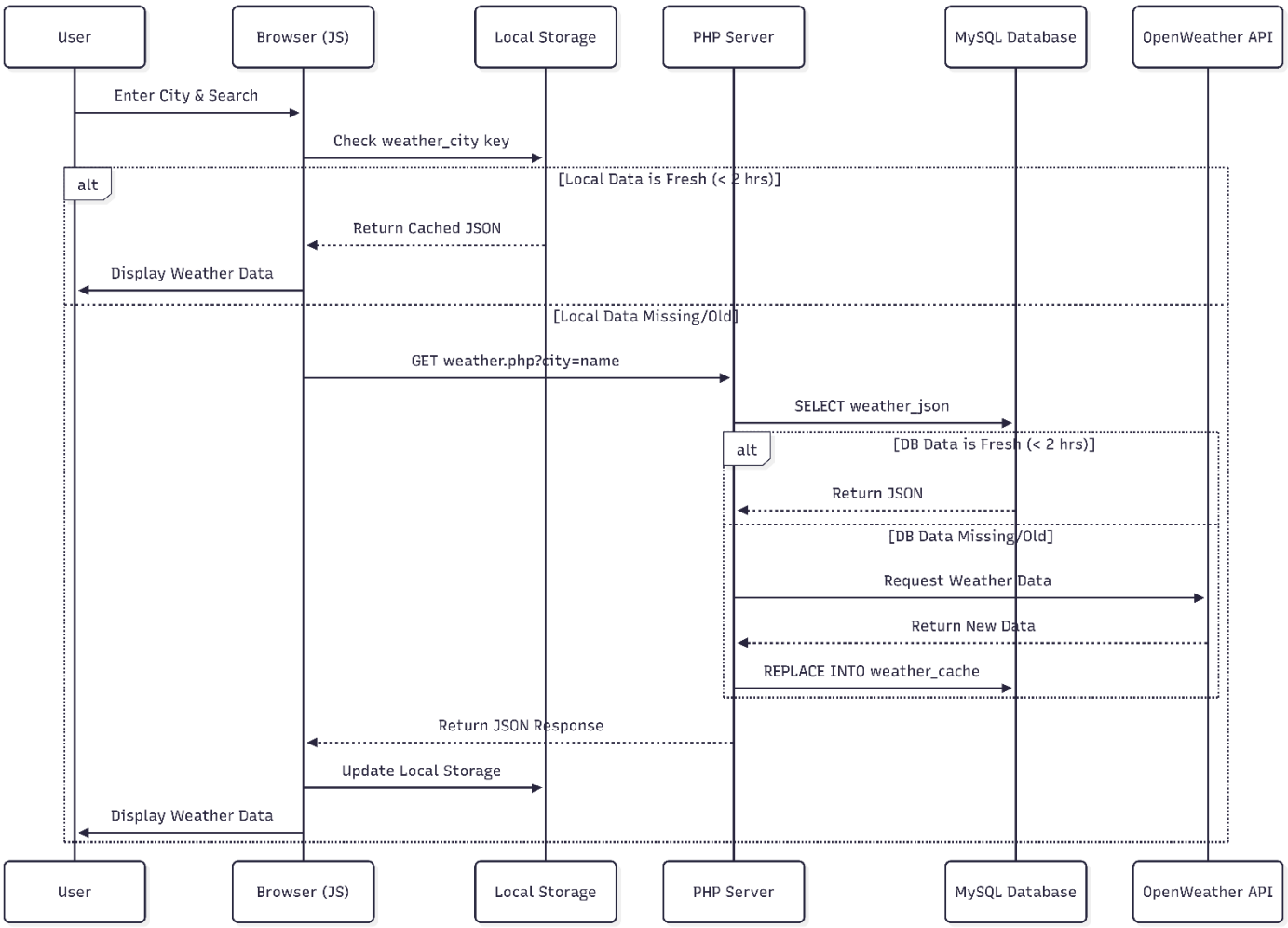
The **design and layout** are also very strong. The interface is clean and uses a dark theme that looks modern. The "Weather Grid" organizes details like humidity, wind, and sunrise times into an easy-to-read format. Additionally, the code includes helpful features that convert raw numbers into things people understand, like changing wind degrees into directions (e.g., "North") and formatting timestamps into readable clock times.

The main weakness is **security**. Even though the code tries to clean the city names, it does not use "prepared statements" for the database. This is a standard safety practice to prevent hackers from attacking the database. Also, the API key (the secret code that lets the app talk to the weather provider) is visible in the code, which is a safety risk.

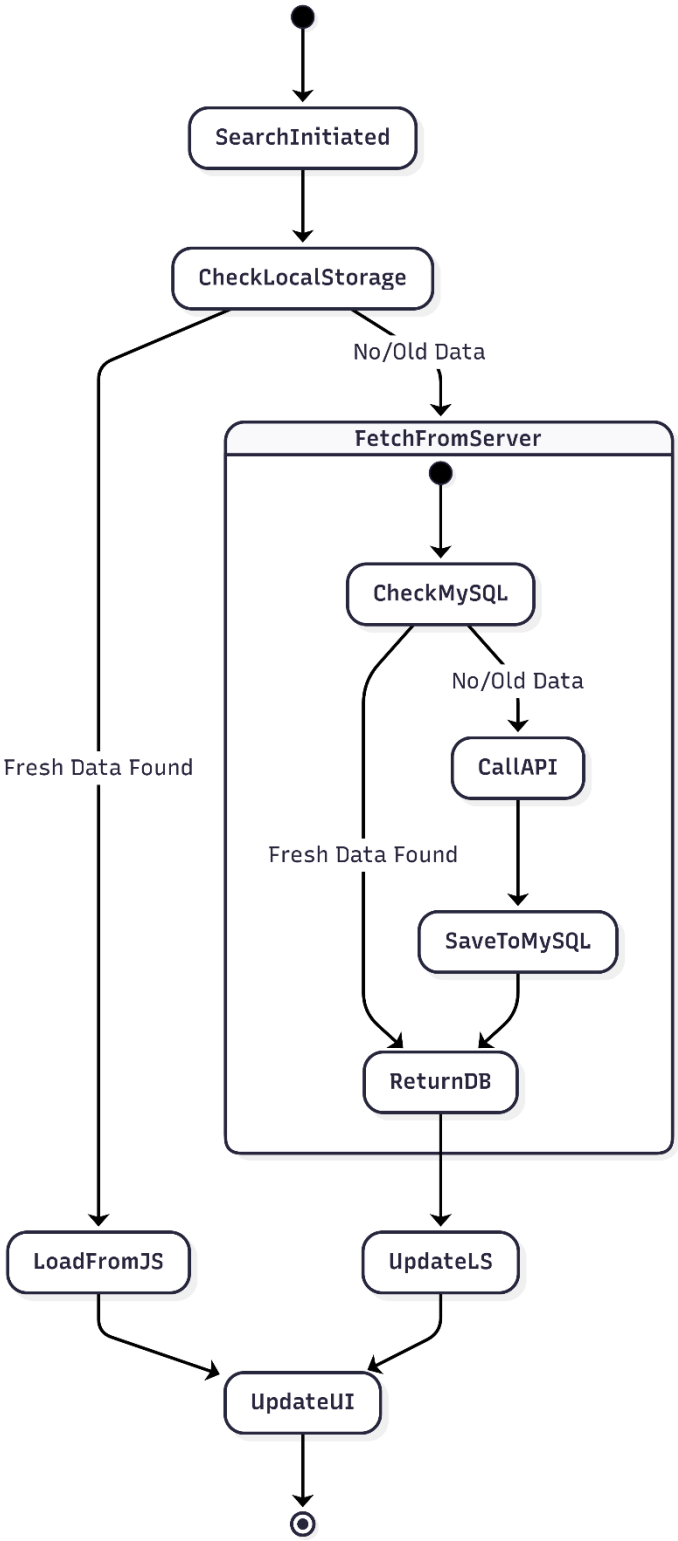
Another issue is **error messaging**. If the internet goes down or the database crashes, the app simply says "City not found." This might confuse users because the city name might be correct, but the connection is broken. Improving these messages would help users understand what is actually happening.

UML diagrams

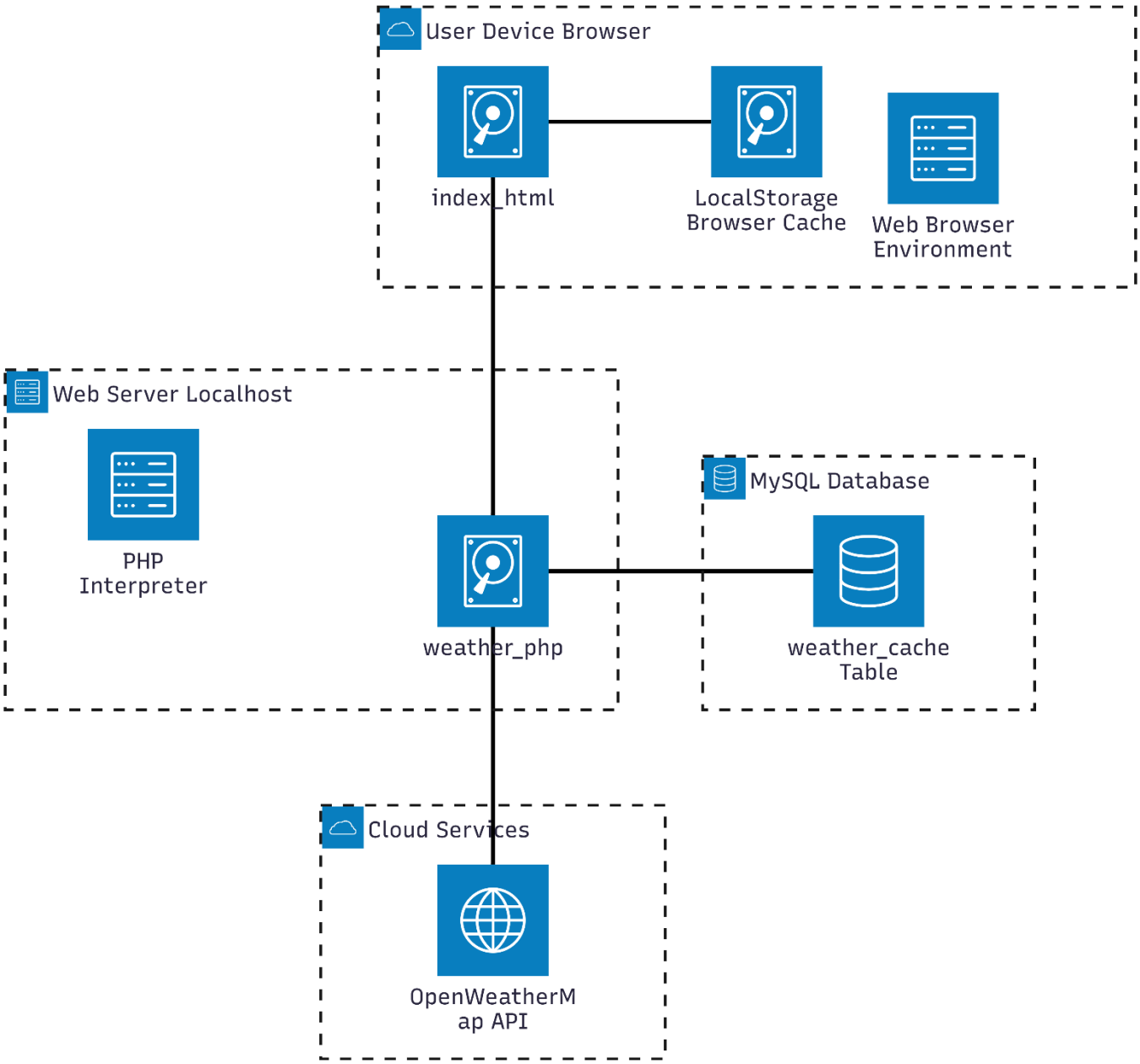
1. Sequence Diagram



2. Activity Diagram



3. UML Deployment Diagram (including the browser cache artefact)



Hosted application link: https://rohit-weather.gt.tc/?i=1